

Shredding to Postgres

How a 1.5 GB JSON corpus becomes four populated Postgres tables. Every stage below is built; the numbers are from running it end to end.

Measured on corpus/ci.json, 1.5 GB Checked with ocamlc 5.5.0

Status complete, loaded and checked

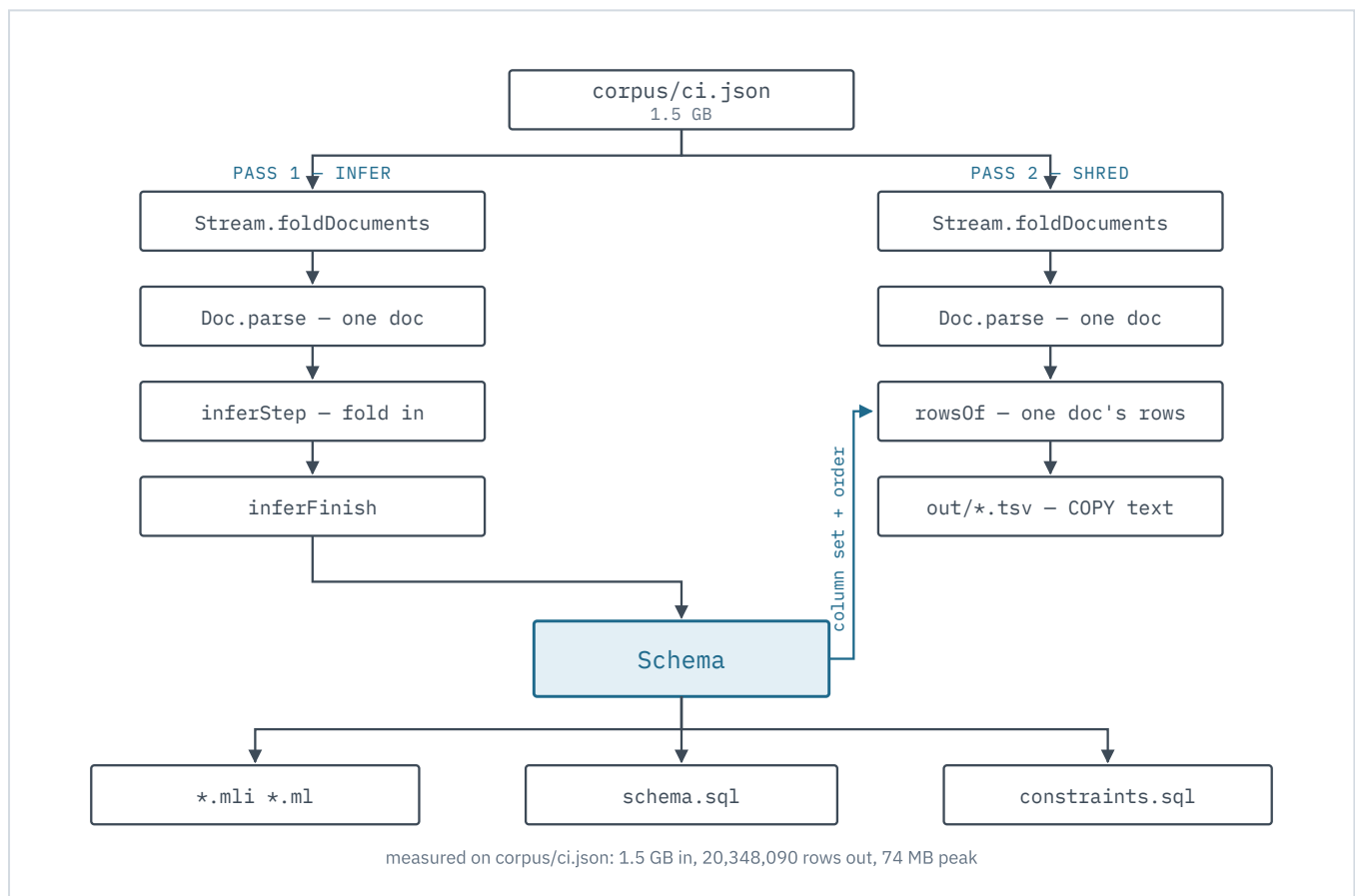
docs/phase_1/pipeline.html ·rebuild with make -C docs pipeline

01

The binary

One command, two passes over the file, and one value — the `Schema` — that everything downstream is derived from.

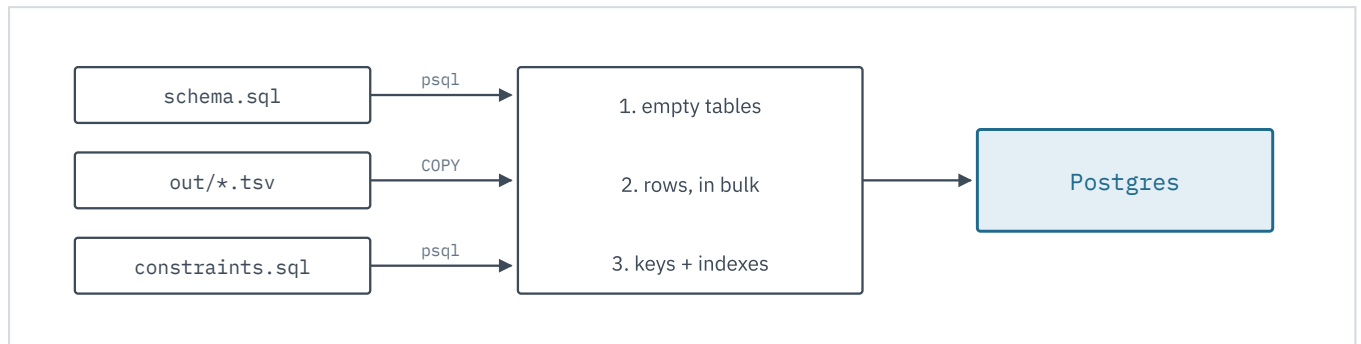
```
tatami --config config.json --data -o out corpus/ci.json
```



Pass 2 reuses pass 1's reader unchanged — the same `foldDocuments`, a different callback. The `Schema` is the only thing that crosses between them, and it is also what makes `schema.sql`'s column order match the `.mli` without anyone maintaining that by hand.

Loading

The binary writes files and stops. Only `psql` touches the database, driven by a generated script — so connection details, Docker and `psql` availability stay out of the generator, and a failed `COPY` leaves something you can read and re-run.



Constraints go on *after* the bulk load, which is what `db/constraints.sql` already does: validating ten million rows one at a time on the way in would dominate the load.

What exists

COMPONENT	WHAT IT DOES	STATE
<code>Stream.foldDocuments</code>	Reads the file in 1 MB chunks, finds one document's bounds in the raw bytes, decodes only that, hands it over, drops it	BUILT
<code>Doc.parse</code>	One document's text to a <code>Doc</code> . Pure and total; streaming did not change it	BUILT
<code>inferStep</code> / <code>inferFinish</code>	Folds each document into the observations; settles absent counts and the end-of-corpus checks	BUILT
<code>toSchema</code>	Observations to the <code>Schema</code> : tables, columns, types, nullability, keys	BUILT
<code>gen</code>	<code>Schema</code> to one <code>.mli</code> and one <code>.ml</code> per table, in compile order	BUILT
<code>ddl</code>	<code>Schema</code> to <code>CREATE TABLE</code> — column order matching the <code>.mli</code> , <code>NOT NULL</code> where there is no <code>option</code>	BUILT
<code>constraints</code>	Foreign keys and the indexes a three-hop join needs	BUILT
<code>rowsOf</code>	One document to its rows: minted keys, <code>idx</code> from array position, <code>key</code> from map entry name	BUILT
<code>TSV writer</code>	<code>COPY</code> text — tab separated, <code>\N</code> for null, backslash escapes, <code>t / f</code> . A number keeps the literal it was written with, so it round-trips exactly	BUILT
<code>load.sh</code>	The three <code>psql</code> steps, with the table list taken from the schema rather than hardcoded	BUILT

End to end, measured

One command over the real corpus, then the generated loader.

1.5 GB IN	ROWS OUT	SHRED	LOAD
74 MB	20,348,090	360 s	109 s
PEAK, BOTH PASSES	ACROSS 4 TABLES	TWO PASSES	COPY + CONSTRAINTS

Memory is constant, not proportional: 73 MB whether the input is 21 MB or 1.5 GB. Before streaming it was about forty-five times the input, which put the full corpus at roughly 67 GB. Loading is the generated `load.sh` against Postgres 16, and every foreign key resolved.

Two facts that decide the shape

The file is read twice

A row cannot be written until the schema is final, and a member first appearing in document 800,000 changes both the column set and its order. So pass 1 must finish before pass 2 starts. Nothing carries between them: keys are minted during pass 2 and nothing in pass 1 depends on them, which is what makes two passes cheap rather than awkward.

The Schema is the only hub

The `.mli`, the `.ml`, `CREATE TABLE`, the constraints and the column order the TSVs are written in all come from one value. That is what keeps the database a faithful second copy of the same data — today that agreement is maintained by hand, in a comment.

Before starting

1. **Do the generated `.ml` files have a job?** Phase 3 parses the `.mli` text and never compiles anything we emit, and effects are cancelled, so nothing currently consumes them. They become the most important file we emit only if `Root.get` is meant to become a Postgres query.
2. **Table naming.** Ours are path-derived — `root`, `runs`, `runs_jobs`, `runs_jobs_steps`. The filename becomes the SQL table name, so this decides the database too.
3. **Column order.** Alphabetical today; document order is what makes `schema.sql` line up with the `.mli`, and what the existing test expects.